impact on live traffic versus usefulness to understand what is going on. The challenge here is twofold:

- To get meaningful insights, you need to load debug symbols, which can and will change the execution times and cause higher resource usage.
- Then, there is the overhead of tracking, ingesting, and storing the profiles (e.g., in `pprof` format). For that, you will need another backend that is able to store profiles efficiently and allows you to query them or one of your existing backends to introduce support for profile handling.

What we see in real life so far is that CP is validated in prod, but there are few best practices to share. Therefore, you would need to decide on a case-by-case basis whether the returns from CP outweigh the investments required (in terms of storage, ingestion, latency impact, etc.).

Summary

- Developer observability means using telemetry signals to equip developers with actionable insights to accelerate development; debug code; or understand the impact of a new feature, in terms of runtime or resource usage.
- You can use continuous profiling to understand the current performance and resource usage of your service as well as compare resource usage with previous versions of your service.
- The protobuf-encoded `pprof` format is the closest we have to a standard for representing profiles on the wire or at rest.
- Flame graphs render call stacks in a visual manner, and eBPF is a Linux kernel extension mechanism used for profile collection across many continuous profiling tools.
- To automate the collection of profiles from either your app code or the operating system, you can use eBPF, increasingly gaining traction.
- Open source continuous profiling tools are a relatively recent development and include Parca, Pixie, and Pyroscope.
- Cloud providers and observability vendors alike provide commercial offerings in the continuous profiling space.
- Developer productivity tools, usually based on OpenTelemetry, provide developers with insights on code changes from a performance and resource usage view.
- There are a few challenges, including a lack of interoperability and the risk of small players being acquired or discontinuing an offer; however, the overall benefit of developer productivity tooling is clear and will likely go mainstream.
- You can benefit from continuous profiling not only in dev environments but also in prod, if you know the impact on the system.
- Support for open standards is crucial. You will have to do your due diligence to make sure your vendor (or open source project) of choice is fully committed to supporting their offering.